

6.2.3 UCS2/UTF-16

16-bit character data. Each 16-bit unit is transmitted with the *b1* “first”, up to *b16* “last”. (NOTE: at a lower protocol level: in 16-QAM 4 bits are sent in parallel, 64-QAM: 6 bits in parallel, 256-QAM: 8 bits in parallel, 1025-QAM: 10 bits in parallel.) Though nominally big-endian, it is bit-wise little endian (so if each octet is ‘reversed’ individually (instead of reversing 16-bit units), the encoding is little-endian octet-wise...).

- Bits per character: 16 (bits per code unit; a character in planes 1-17 are coded by 2 16-bit code units; the byte order is big-endian if 16-bit units ‘reversed’, little-endian if 8-bit units ‘reversed’)
- Pad character: CR (as 0x000D in 16 bits)
- Character table: ISO/IEC 10646 [10]; UTF-16, 1 to 2 hexadecets per character

UTF-16 is an extension of UCS2, plane 0, that allows for representing characters in planes 1-16.

When splitting a message text into submessages (using this encoding), there must be no cut inside of a 16-bit unit (if UCS2 is supported; since the max number of message bytes is even for each submessage both for SMS and CBS, this will not happen), nor between “high surrogate” and a “low surrogate” (the pair encodes for a character in plane 1 to 17; if UTF-16 is supported). (There should be no “surrogate” not in such a pairs; that would be an encoding error.)

Note that for CBS, a 2-letter language tag prefix for UCS2/UTF-16 encoded messages are encoded in the 7-bit 0x00 alphabet in the first two bytes; it should be lowercase.

Furthermore, if any of the 7-bit alphabets 0x10 and onwards can be used for an SMS/CBS message, use of that 7-bit alphabet is preferred, as that can more than half the length (in bytes) of the message, allowing for fewer submessages and allowing a longer (in characters) total message. (Note that alphabets 0x00-0x0F are deprecated for SMS/CBS messages.)

6.2.3.1 UCS to UCS mapping

For messages originally encoded in a UCS encoding, the following mappings should be used, both upon sending (before encoding in a 7-bit encoding, if that is to be used), as well as upon receiving a UCS2 SMS/CBS message:

- U+0000-U+0008, U+000E-U+001F, U+007F-U+009A, U+009C-U+00A0: map to U+FFFD.
- U+0009: map to U+0020. U+000B, U+2028-U+2029: map to U+000A. U+061C: map to U+200F.
- U+2000-U+200B: map to U+FFFD.
- If in the immediate vicinity of an Arabic letter: U+200C: map to U+00B7 (use middle dot as non-joiner). U+200D: map to U+0640 (use kashida as joiner). Note that U+200D and U+200C have special handling for several Indic scripts, including when converting to 7-bit alphabets (see section A.4).
- U+202A-U+202E, U+205F-U+206F, U+D800-U+F8FF, U+FDD0-U+FDEF, U+FEFF, U+FFFC, U+xFFFE-U+xFFFF (where *x* is in {0, ..., 10₁₆}): map to U+FFFD.

6.2.3.2 Bidirectional scripts

In case there are any bidi RTL characters in the message, a *simplified* Unicode bidi algorithm should be used before display of the message. Bidi AL is handled as bidi R, except Arabic punctuation which is handled as ON; all bidi EN and AN handled as bidi L; all bidi ET, ES, CS handled as bidi ON. This is not limited to SMS/CBS.

Also, the ‘paragraph direction’ is always LTR for SMS/CBS messages. Note also that there are no nestable bidi controls in SMS/CBS (or they are not interpreted in the simplified bidi algorithm, in case the mappings above have not been applied); for SMS/CBS bidi controls should be mapped to U+FFFD (see 6.2.3.1).

Furthermore, Arabic is a ‘cursive’ script and Arabic characters thus should be cursive shaped (initial, medial, final, isolated forms). Both Arabic and Hebrew may have (certain) combining characters applied (like for other scripts as well). Bidi mirroring (when displaying RTL text sections) applies, but (simplified for SMS/CBS) *only* those that can be done via using the data in BidiMirroring.txt (Unicode) (not the comments in that file).

6.2.3.3 Complex scripts

Brahmic (Indic) scripts (if supported) also should be handled specially for rendering; the details of which are beyond the scope of this standard. Further, while most ‘emoji’ characters are isolated small, coloured drawings, many emoji are made from ‘emoji sequences’, multiple emoji characters creating one emoji to display.

For the Latin script alphabet (“European”, whether using the alphabet with identifier 0x10 or UCS2/UTF16) most text for “European” languages (and Vietnamese) can be handled by “precomposed” letters in ISO/IEC 10646 (UCS). However, to be able to fit the Latin characters for these languages into one 7-bit alphabet (7-bit alphabet identifier 0x10), we rely on combining characters, in particular for East European languages and Vietnamese. Similarly for polytonic Greek (modern Greek orthography is monotonic Greek, and that is covered without the need for combining characters in the 7-bit alphabet for Greek, identifier 0x11). The easiest way to handle that is to upon reception of a message in Latin (0x10) or Greek (0x11) 7-bit alphabet, is to first convert to ISO/IEC 10646 and then apply the Unicode Normal Form C (NFC). Upon sending, first apply NFC, then decompose letters that need to be represented in decomposed form in the Latin or Greek 7-bit alphabet.

6.2.3.4 Quote mark (line-break:QU) disambiguation for bidi and automatic line break

Among begin-end punctuation, many quote marks are, in typographic tradition, ambiguous. It is both (loosely) language dependent, and in some cases the same mark is used both for start and end quote. We cannot change typographic tradition here (though it may evolve independently). The best we can do, for two important display processes, is try to infer (ad hoc) which instances of ambiguous quote marks that are begin quote marks and which are end quote marks. Unfortunately, the Unicode standard does not do (as per 2026) this even close to correctly (and for bidi, not at all). So we need tailoring of both the (Unicode) bidi algorithm and the (Unicode) line breaking algorithm, so that bidi reordering and line breaking perform reasonably well. There is no semantic analysis here (nor in the Unicode algorithms), and people do not always follow writing conventions, so the best we can do is an ad hoc analysis assuming that writing conventions are followed sufficiently well. The bidi algorithm (section 3.1.3 Paired Brackets) loosely assumes that there is bracket ‘pairing’, presumably (as of 2026, not clearly stated in the bidi algorithm) trying to match with a ‘mirror’ bracket character; presumably (again) including nest counting.

For ambiguous quote marks, we will assume the following kind of (non-mirror...) matching: open quote marks that are comma-like and double have as (fake mirror) match a close quote mark that is comma like and double; similarly for comma-like and single; similarly for comparison-operator-like and double; similarly for comparison-operator-like and single. Note that in an actual text, brackets might not always have open-close matching, and it is implementation defined how that is handled. (In particular, ‘)’ may be used a list number indicator, and not a bracket at all; detecting that as well as other border-line cases (like the quoting in this sentence), including typos, requires some level of semantic analysis, which is out of scope.)

This ad-hoc analysis (given below) and begin-end matching (as above) must be done before the paired brackets handling (section 3.1.3 Paired Brackets) in the bidi algorithm, which in turn is done before the automatic line breaking processing.

The ad hoc inference is for most scripts based on that words are separated by space, as it is in most scripts. For scripts that do not use space between words, in particular Khmer, Thai, Lao, and Chinese and Japanese, we use a simpler ad hoc, close to the Unicode approach for line breaking. Note that modern Korean does use space between words, and that Chinese and Japanese conventionally use other quote marks that are not ambiguous (i.e. not QU).

6.2.3.4.1 QU ad hoc disambiguation for scripts using space between words

If a line of text (before automatic line breaking) has one or more QU characters:

If a) immediately *preceded* by beginning of line, an OP (bidi-bracket:o, open), or an SP; and b) immediately *succeeded* by a non-SP; then regard the QU character as an OP (bidi-bracket:o, open).

If a) immediately *preceded* by a CL (bidi-bracket:c, close), general category L* or N*, or .?! (or reverse ?); and
b) immediately *succeeded* by end of line, a CL (bidi-bracket:c, close), an SP, or .?!; (or reverse ?;);

Repeat this on the line containing QU until no change (a QU may change to OP or CL, and then another QU may be disambiguated).

Ideally, all QU in the line should then be disambiguated (even the ASCII ones " '). But in case there is still any remaining QU characters, regard them as line-break:SY (and not bidi-bracket).

This disambiguation is ad hoc and rely on authors abiding by common writing conventions. However, this tailoring is better than status quo (2026) for Unicode line breaking and Unicode bidi.

6.2.3.4.2 QU ad hoc disambiguation for scripts not using space between words

Thai, Lao and Khmer do not use space between words (but may use it between ‘phrases’, like comma). Also Chinese and Japanese do not use spaces between ‘words’, but in these scripts, one normally do not use any QU quote marks, but instead Chinese/Japanese quote marks that are not QU, but already OP or CL.

For these scripts we cannot rely on SP for disambiguation of QU, and indeed when used in those scripts, they appear to be used in accordance with their Unicode general category unambiguously. Thus:

A QU that is Pi, regard it as OP (and Ps; bidi-bracket:o, open). A QU that is Pf, regard it as CL (and Pe; bidi-bracket:c, close). Other QU (in particular, ASCII quote marks, " and '), regard as SY.

6.2.3.5 Automatic line breaking

There are a few more things amiss with status quo (2026) for Unicode line breaking (in addition to needing better disambiguation for QU characters).

SP: A non-empty sequence of SP always, with *one* exception, offer a line break opportunity after the sequence. The exception: for convenience, a single SP between two Nd (decimal digits) offers *no* line break opportunity. This exception is due to the commonality of using a space as thousands separator; though PUNCTUATION SPACE (which is not a line-break:SP) would be ideal, but in practice unused. Thus, SP before !?;» and after «*do* offer a line break opportunity after the SP. There is no special treatment for fr_FR (the only special case locale regarding this) and 'two-part' punctuation, like ?!;«» (two-part punctuation, the only special cases for fr_FR). Use NARROW NO-BREAK SPACE for no-break spaces in this case.

Final punctuation, here ;,.,?! and reverse ,;?, plus CL (mainly }]) and CL resolved QU): A (commonly mixed) non-empty sequence of these offers a line break opportunity after the sequence. However, with two exceptions: 1) a single , or . between two decimal digits (Nd) offers *no* line break opportunity after the comma or full stop, and 2) a single letter (L*), followed by a colon or full stop, followed by a single letter, offers no line break opportunity after the colon or full stop. (For Brahmic scripts (Indic and more) also ‘danda’ and ‘double danda’, not in common modern use, are final punctuation.)

Solidus (and reverse solidus): A / or \ that is immediately preceded by at least two general category L* characters, immediately succeeded by at least two L* characters, offers a line break opportunity after the / or \. This way there is a line break opportunity when / is used as “or”, as well in (most cases) path names (incl. URLs), which may be quite long without other break opportunities, still without any break in abbreviations like n/a, nor in number divisions like 16/20.

6.2.3.6 CSI control sequence compression

.....

6.2.3.7 Simple tables via SMS-specific use of IS1..IS4

.....